iLearn Practice Exam

12BR · Chapter / Topic 19

Cambridge A Level Computer Science 9618

Paper 3 Advanced Theory

Topic 19: Computational Thinking and Problem-solving

Name: _____

Class: _____

Date: _____

Time allowed: 55 minutes

TOTAL MARKS: 46

Marks obtained: ..... / 46

INSTRUCTIONS

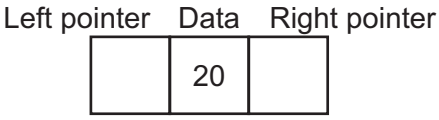
· Answer all questions.
· Write your answers in the spaces provided on this paper, using a black or dark blue pen.
· Do not use an erasable pen or correction fluid.
· No calculators are allowed.
· Do not write anything in the margins.

INFORMATION

· The total mark for this paper is 46.
· The number of marks for each question or part question is shown in brackets [ ].
· This paper contains 7 questions, all drawn from Cambridge past papers on this chapter.
· After you finish, your teacher will mark your work against the official mark scheme.
· Only 46 marks of past-paper questions are available for this chapter, so the recommended time is 55 minutes (1 min 1

Questions reproduced from Cambridge International past papers © Cambridge University Press & Assessment, for classroom practice.

1 **(a)** A linked list of nodes is used to store an ordered list of integers. Each node consists of the data, a left pointer and a right pointer, for example:

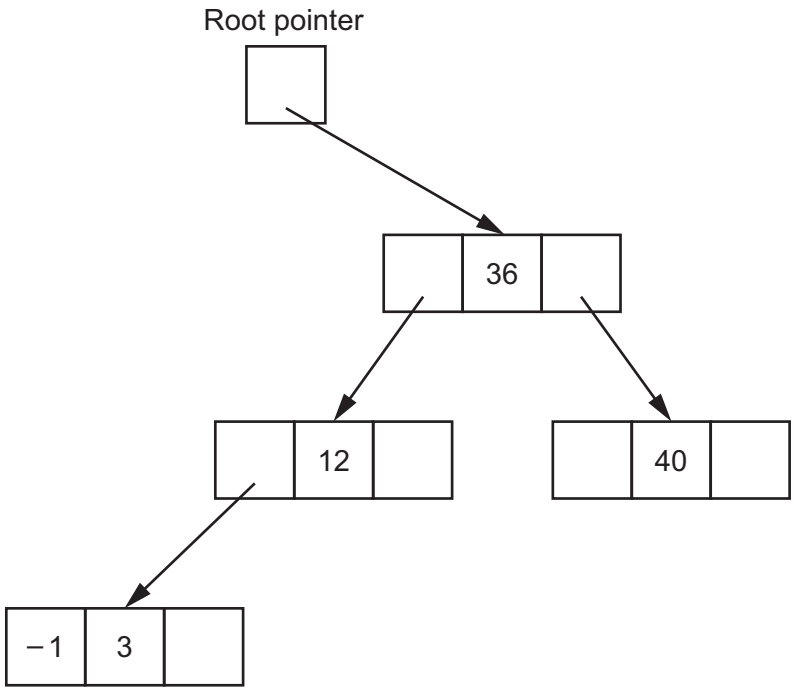


The linked list will be organised as a binary tree.

−1 is used to represent a null pointer.

Complete the binary tree, including null pointers, to show how the data will be organised after the following integers have been added:

6, 15, 41, 66



[4]

(b) Describe what is meant by recursion.

.....

.....

.....

..... [2]

(c) A binary tree is a suitable Abstract Data Type (ADT) that a designer can implement using recursive algorithms.

Identify **one other** ADT that a designer can implement using recursive algorithms.

..... [1]

2 Complete the following paragraph using the most appropriate words from the given list.

Some words are **not** used.

adjacent	**Application**	**BitTorrent**	**corresponding**
Internet	**layer**	**Link**	**lower**
protocol	**route**	**Transport**	**upper**

The implementation of a ..... suite can be viewed as a stack. Each ..... of the stack has its own specific functionality and can only communicate with ..... layers of the stack. User interaction is associated with the highest level of the stack, which is the ..... layer. [4]

3 **(a)** A program declares a 1D array globally as `PartName[1:1000000] OF STRING`, and sorts its contents in ascending alphabetical order. The function `PartSearch` performs a binary search on the array. The item to be found is passed as a parameter. If it is found, its location in the array is returned, otherwise, –1 is returned.

(i) Complete the following table to define the variables used in the function.

identifier	data type	description
`Left`		
`Right`		
`Mid`		
`Target`		

[2]

(ii) Complete the pseudocode for the function to perform a binary search on `PartName` using all the variables from the table in **10(a)(i)**.

Some lines may be left blank.

```
FUNCTION PartSearch(.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
ENDFUNCTION
```

[5]

(b) Describe how the time efficiency of a binary search changes as the number of items in the search list increases. Refer to Big O notation in your answer.

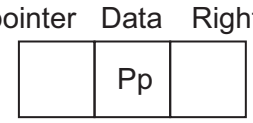
.....

.....

.....

..... [2]

4 **(a)** A linked list of nodes is used to store an ordered list of strings. Each node consists of the data, a left pointer and a right pointer.

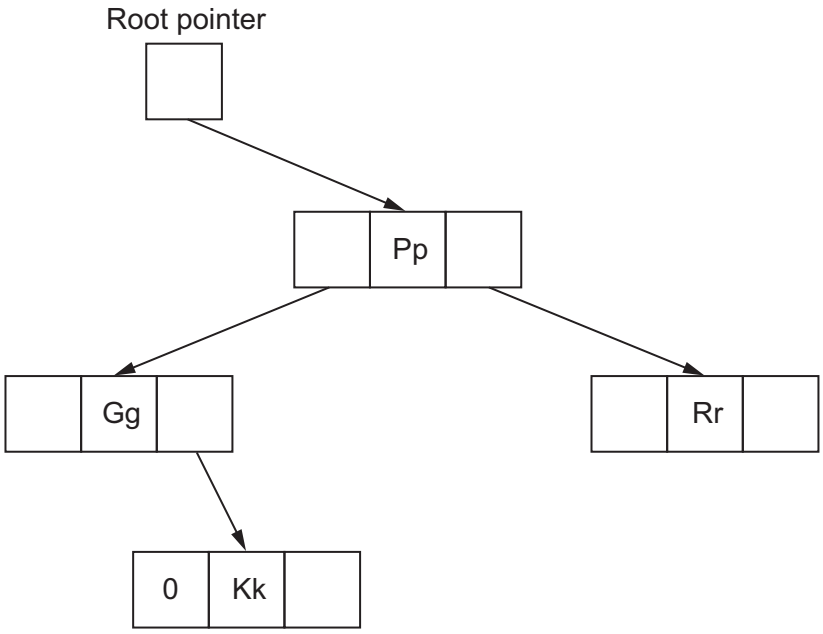


The linked list will be organised as a binary tree.

0 is used to represent a null pointer.

Complete the binary tree, including null pointers, to show how the data will be organised after the following strings have been added:

Aa, Mm, Ss, Xx



[4]

(b) A binary tree can be used to implement recursion.

Identify **one** feature of an algorithm that makes it beneficial to use recursion.
Give **one** example of an application that could use a recursive algorithm.

Feature .....

.....

Example .....

[2]

5 **(a)** An insertion sort algorithm is used to sort the unique ID numbers of club members into ascending order. These unique ID numbers are kept in a 1D array `MemberID`. The corresponding names of the members are stored in a 1D array `Names`.

The arrays are declared as:

```
DECLARE MemberID : ARRAY[0:499] OF INTEGER
DECLARE Names : ARRAY[0:499] OF STRING
```

The following table defines the variables to be used in this algorithm.

identifier	data type	description
`Count`	integer	main loop counter to iterate through the whole array
`Temp1`	integer	temporary holding location for current member ID
`Temp2`	string	temporary holding location for current member name
`Insert`	integer	counter to show current insertion position

Write the pseudocode for an insertion sort to arrange the unique ID numbers into ascending order. Make sure that the data in both arrays is swapped as required so that data with the same index continues to apply to the same person.

Use the given identifiers. Assume all variables have been declared.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

(b) Describe the factors, other than changing the algorithm, that affect the time efficiency of the insertion sort algorithm.

.....

.....

.....

.....

.....

..... [3]

(c) State the Big O notation for the worst-case time complexity of the insertion sort.

..... [1]

6 **(a)** An array is an Abstract Data Type (ADT).

Identify **two** other ADTs.

1 .....

2 .....
[1]

(b) A 1D array `DataArray` holds up to 1000 elements of type integer and needs to be sorted in ascending order.

Write the **pseudocode** for an insertion sort to sort the array into ascending order.

Use the identifiers from the table in your algorithm.

You do **not** need to declare any arrays or variables for this algorithm. You may assume this has already been done.

Identifier	Data type	Description
`Index`	`INTEGER`	counter for outer loop
`Position`	`INTEGER`	counter for inner loop – insertion position
`DataArray`	`INTEGER`	1D array to store up to 1000 integers
`Value`	`INTEGER`	value to insert

The first line has been written for you.

```
FOR Index ← 2 to 1000
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [4]

(c) Describe **two** ways in which the performance of a sort routine is affected by the data to be sorted.

1 .....

.....

2 .....

.....

[2]

7 The recursive procedure `Delete()` is defined as follows:

```
PROCEDURE Delete(Index, Target)
   IF Numbers[Index] > 0 THEN
      IF Numbers[Index] >= Target THEN
         Numbers[Index] ← Numbers[Index + 1]
      ENDIF
      Index ← Index + 1
      CALL Delete(Index, Target)
   ENDIF
ENDPROCEDURE
```

An array `Numbers` is used to store a sorted data set of non-zero positive integers. Unused cells contain zero.

The contents of the array at the start of the algorithm are:

Numbers									
[1]	**[2]**	**[3]**	**[4]**	**[5]**	**[6]**	**[7]**	**[8]**	**[9]**	**[10]**
2	3	7	11	15	17	19	23	0	0

Complete the trace table for the algorithm for the procedure call:

```
CALL Delete(1, 15)
```

		Numbers									
Index	**Target**	**[1]**	**[2]**	**[3]**	**[4]**	**[5]**	**[6]**	**[7]**	**[8]**	**[9]**	**[10]**
		2	3	7	11	15	17	19	23	0	0

[4]

END OF QUESTION PAPER